Lecture 24: The thinking revolution

PSYC 51.17: Models of language and communication

Jeremy R. Manning
Dartmouth College
Winter 2026

Learning objectives

By the end of this lecture, you will be able to...

1. Explain **chain-of-thought prompting** — why intermediate reasoning steps improve LLM performance
2. Describe **how reasoning models are trained** via reinforcement learning on verifiable rewards
3. Explain the **mechanics of test-time compute scaling** — what happens inside a reasoning model at inference
4. Analyze **why thinking works**: the relationship between token generation and computation
5. **Evaluate the evidence**: is longer "thinking" genuine reasoning or sophisticated pattern completion?
6. Explain **Mixture of Experts** — how sparse activation lets models store more knowledge while keeping inference costs manageable

Welcome back!

The last content week

This is our final week of new material. Three (content) lectures remain:

- **Today**: The thinking revolution — how and why reasoning models work
- **Wednesday**: Agents, tools, and the agentic era
- **Friday**: The reckoning — society, safety, and what comes next

Final project reminder

Final Project presentations are on **March 9** (next Monday, our final day of class). Deliverables:

- A **notebook** containing your project's code, results, and analysis (should run in Google Colab)
- A **presentation** (presented *in class* on March 9) summarizing what you did, what you found, and why it matters. You should also upload your slides to Canvas.
- A **final report** (roughly 2—5 pages) describing your project in detail

Optional help session

I can be available during our X-hour (**Thursday**) to help with final projects, if there is interest in this. I'll treat it as an open Q&A session; I'll plan to stay, answer questions or help on a first-come-first-served basis, until there are no more questions, and then we'll wrap up.

A new paradigm: test-time compute

A new scaling axis

Every model we've studied so far (rules-based models, embeddings, autoregressive models, and diffusion models) improved primarily by making **training** bigger: more data, more parameters, more compute during training. In late 2024, a new paradigm emerged: **test-time compute scaling** — spending more compute *at inference* to improve results.

The key insight

A smaller model that "thinks longer" can (sometimes) outperform a larger model that answers immediately. This decouples model quality from model size in a way that changes how we build and deploy AI.

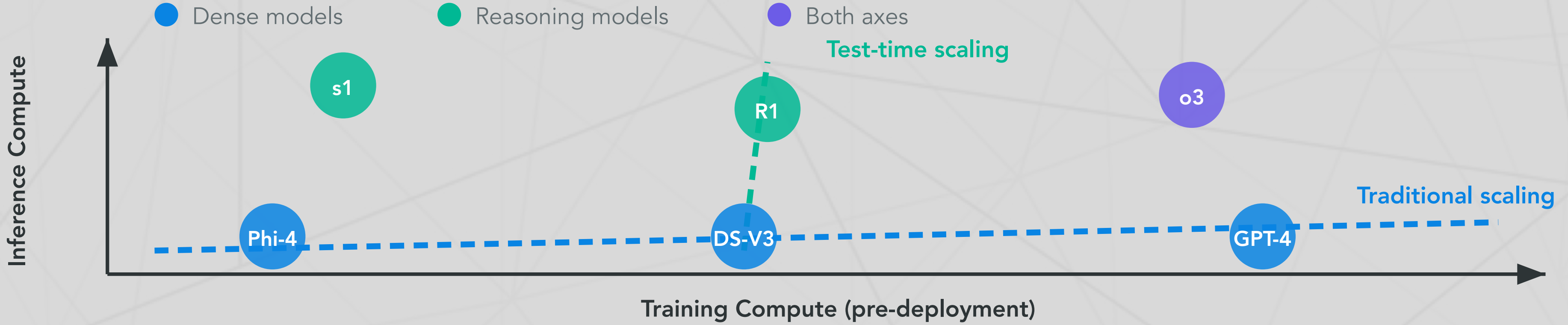
Follow along!

Check out some interactive examples in the companion notebook.

Two scaling axes

Training compute vs. inference compute

	Training compute	Inference compute
When	Before deployment (once)	At query time (every call)
What improves	Base knowledge, capabilities	Reasoning depth on hard problems
Scaling law	Kaplan et al. (2020)	Snell et al. (2024)



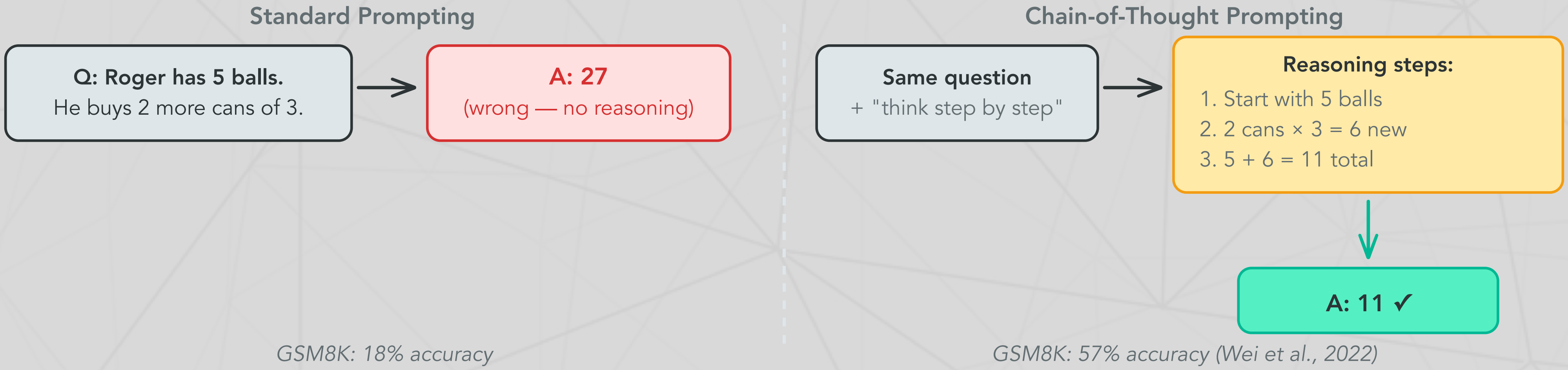
Key terms

- **Traditional scaling**: improving models by increasing training data, parameters, and compute (Kaplan et al., 2020)
- **Test-time scaling**: improving outputs by spending more compute *at inference* (Snell et al., 2024)
- **Scaling law**: a power-law relationship between compute invested and model performance
- **"Both axes"** (purple in chart): models like o3 that invest heavily in *both* training and inference compute

Chain-of-thought prompting

Wei et al. (2022, NeurIPS)

Chain-of-thought (CoT) prompting showed that LLMs perform dramatically better on reasoning tasks when prompted to **generate intermediate steps** before answering. This requires no model changes — just different prompting.



Why does this work?

Each generated token is a **computation step**. When a model writes "2 cans × 3 = 6," it's *predicting* "6" as the most likely next token based on patterns learned during training — and that prediction then gets stored in context, where subsequent tokens can build on it. More intermediate tokens = more serial computation = harder problems become solvable.

What is GSM8K?

GSM8K (Cobbe et al., 2021) — **Grade School Math 8K**, a dataset of 8,500 grade-school math word problems requiring 2–8 reasoning steps using basic arithmetic. A standard benchmark for multi-step mathematical reasoning in LLMs.

Why do intermediate steps help?

Key terms: threshold circuits

- **Threshold circuit**: a Boolean circuit whose gates are *threshold functions* — a gate outputs 1 if at least k of its n inputs are 1. These circuits can compute addition, multiplication, and sorting.
- **Constant-depth** (TC^0): a threshold circuit with a *fixed* number of layers, no matter the input size. It can do a lot in parallel but has limited sequential depth.
- **Why this matters**: a transformer with L layers is essentially a constant-depth threshold circuit — it always performs exactly L sequential steps per token, regardless of problem difficulty.

Transformers are constant-depth circuits

A transformer with L layers performs L sequential computation steps per token. For a 100-layer model answering in one token, you get **100 steps of computation** — regardless of problem difficulty.

But if the model generates N intermediate tokens first, it gets $L \times N$ **steps** — the entire model runs once per token, and each token can attend to all previous tokens.

The formal connection

Merrill & Sabharwal (2024) proved that constant-depth transformers are limited to problems in the complexity class TC^0 (constant-depth threshold circuits). But with a chain-of-thought of length T, a transformer can simulate T steps of any Turing machine — making it **Turing-complete**.

In plain language: without CoT, transformers literally *cannot* solve certain problems no matter how large. With CoT, they can solve *anything* (given enough tokens).

From prompting to training: the evolution

The evolution

Year	Technique	Key idea	Reference
2022	Chain-of-thought prompting	Prompt the model with reasoning examples	Wei et al.
2023	Tree-of-thought, self-consistency	Generate multiple paths, pick the best	Yao et al.
2024	**Reasoning models** (o1)	Train the model via RL to reason on its own	OpenAI
2025	Adaptive thinking (Claude 4.x)	Model decides *when and how much* to think	Anthropic

The conceptual leap

CoT prompting (Wei et al., 2022) showed that intermediate reasoning helps. Reasoning models take this further: instead of *prompting* the model to reason, you **train it via reinforcement learning** on verifiable rewards (correct math, passing code tests). The model learns to generate its own internal reasoning traces — and these traces can be far more effective than human-written examples.

How reasoning models are trained

The RL training loop

The key innovation: instead of supervised learning (human writes the reasoning), use **reinforcement learning** where the model discovers *its own* reasoning strategies:

1. **Sample**: Model generates multiple complete solutions (reasoning + answer) for each problem
2. **Verify**: Check which answers are **correct** (math: exact match; code: passes unit tests)
3. **Update**: Reinforce reasoning patterns that led to correct answers; suppress those that didn't
4. **Repeat**: Over millions of problems, the model learns which reasoning strategies work

The crucial requirement

This only works for **verifiable** tasks — problems where we can automatically check correctness. Math, coding, and formal logic have clear right/wrong answers. Open-ended writing, ethics, and creative tasks do not. This is a fundamental limitation of the RL approach to reasoning.

GRPO: how DeepSeek-R1 learns to reason

Group Relative Policy Optimization (GRPO)

DeepSeek-R1 uses **GRPO** (Group Relative Policy Optimization) — a simpler alternative to **PPO** (Proximal Policy Optimization, the standard RL algorithm used for RLHF, which requires a separate "critic" model to estimate value). GRPO eliminates the critic by comparing solutions within each group:

1. For each problem, generate a **group** of G candidate solutions (e.g., $G = 64$)
2. Score each: correct answer → reward $+1$, wrong → reward 0
3. Compute **relative advantage**: how much better/worse than the group average?
4. Update the model to increase probability of above-average solutions

Concrete example

```
Problem: "What is 17 × 23?"
  Solution 1: "17 × 23 = 17 × 20 + 17 × 3 = 340 + 51 = 391" √ → reinforce
  Solution 2: "17 × 23 = 17 × 25 - 17 × 2 = 425 - 34 = 391" √ → reinforce
  Solution 3: "17 × 23 = 300 + 91 = 391"                    √ → reinforce (less)
  Solution 4: "17 × 23 = 381"                               ✗ → suppress
```

What emerges from RL training

DeepSeek-R1-Zero: zero supervised fine-tuning

DeepSeek-R1 was fine-tuned with RL only — no human-written reasoning examples at all. The reward signal was *solely* whether the final answer was correct. Yet the model spontaneously developed:

- **Self-verification**: "Let me check this... wait, that's wrong"
- **Backtracking**: "Actually, I should try a different approach"
- **Structured reasoning**: Breaking problems into numbered sub-steps
- **Reflection**: "This seems too easy, let me double-check"

Why this is remarkable

These reasoning strategies were **never demonstrated** to the model. They emerged purely because they lead to more correct answers. The model "discovered" that doubting itself, re-examining its work, and trying alternative approaches is useful — through optimization pressure alone.

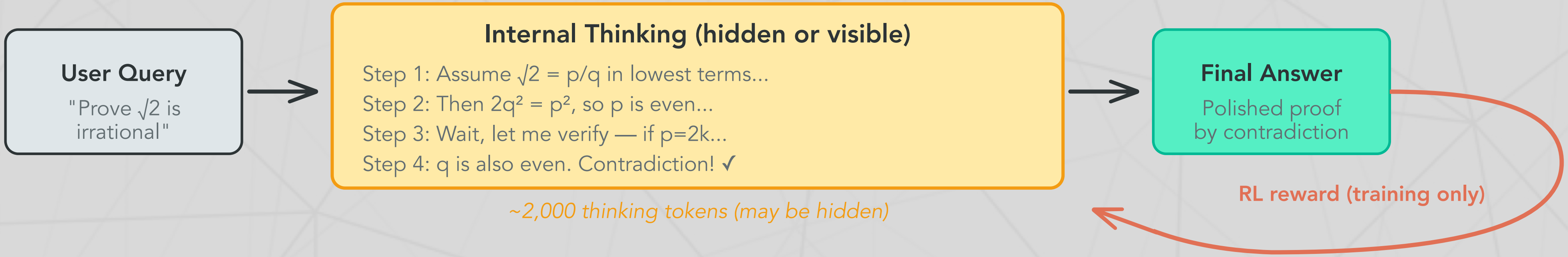
What is AIME?

The American Invitational Mathematics Examination is a 15-question, 3-hour competition for top high school math students. Problems cover algebra, geometry, number theory, and combinatorics. Each answer is an integer 0–999. It is widely used as a benchmark for AI mathematical reasoning.

DeepSeek-R1's performance

DeepSeek-R1's performance on the AIME jumped from **15.6%** (base model) to **71.0%** (after RL training).

The reasoning model pipeline



What happens at inference

1. **User sends a query** — the model begins generating "thinking tokens"
2. **Internal reasoning** — the model works through the problem step-by-step, potentially backtracking and self-correcting (this is where the extra compute goes)
3. **Final answer** — a polished response generated after reasoning is complete
4. **RL reward loop** (during training only) — correct answers reinforce the reasoning patterns that produced them

What thinking tokens look like in practice

Thinking vs. output tokens

When a reasoning model processes a query, it generates two types of content:

1. **Thinking tokens** — internal reasoning (may be hidden or visible depending on the provider)
2. **Output tokens** — the final answer shown to the user

Example: visible reasoning trace

```
[THINKING]  Assume √2 = p/q, coprime. Then 2q²=p²,
            so p even. Let p=2k → q also even.
            Contradiction! √
[OUTPUT]    Proof: √2 is irrational by contradiction...
```

Key differences across providers

	OpenAI o-series	Claude	DeepSeek-R1
Thinking visibility	Hidden (summarized)	**Visible** via API	Visible (open-weight)
Control	`reasoning_effort`	`budget_tokens`	Token count
Thinking tags	Not exposed	`thinking` block in response	`<think> … </think>` tags

The s1 experiment: reasoning is surprisingly simple

Muennighoff et al. (2025)

The s1 model showed that test-time scaling can be achieved with remarkably little effort:

1. Fine-tune Qwen2.5-32B on just **1,000 curated examples** — the "s1K" dataset, a curated set of 1,000 challenging math, science, and coding problems with detailed reasoning traces
2. At inference, apply **budget forcing**: when the model tries to stop reasoning, append the token "Wait" to the model's own output, forcing it to continue thinking rather than giving a final answer
3. Result: Exceeds o1-preview (a much larger model, trained using human feedback) on AIME 2024 (the 15-question high school math competition from a few slides ago) by up to **27%**!

The implication

You don't need massive RL infrastructure to get reasoning capabilities. A small amount of high-quality reasoning data + a simple inference trick can unlock substantial test-time scaling. This suggests reasoning is *latent* in large pretrained models — it just needs to be **activated**.

Claude's extended thinking

Anthropic (February 2025 — present)

Anthropic implemented reasoning as a **toggle within a single model** — same weights, different inference behavior. Unlike OpenAI, Claude's thinking tokens are **visible** to developers and can be controlled using their API.

API usage

```python
from anthropic import Anthropic

client = Anthropic()  # Reads ANTHROPIC_API_KEY from environment
response = client.messages.create(
    model="claude-sonnet-4-6-20250514",
    max_tokens=16000,
    thinking={"type": "enabled", "budget_tokens": 10000},
    messages=[{"role": "user", "content": "Prove that √2 is irrational."}]
)
# Response includes a visible "thinking" block + final answer
```

Key differences from OpenAI

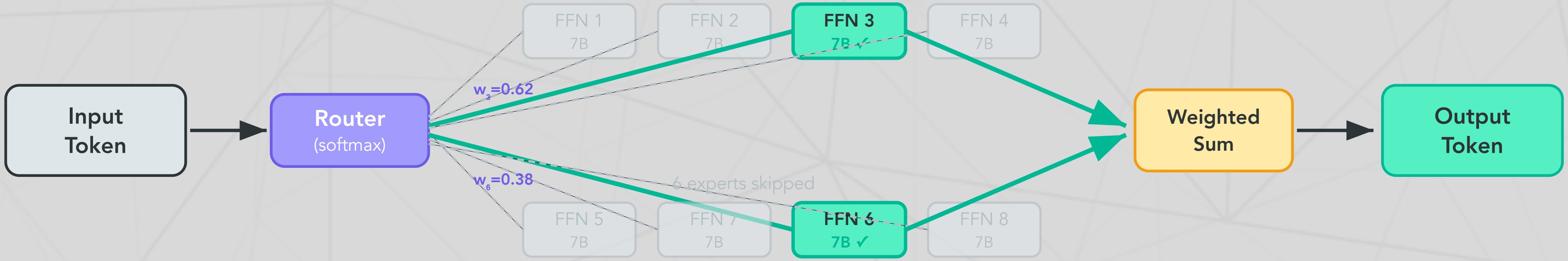
Feature	OpenAI o-series	Claude extended thinking
Thinking visibility	Hidden	**Visible**
Control mechanism	Reasoning effort (low/med/high)	Budget tokens (1K–128K) or adaptive
Latest innovation	o4-mini: tools in reasoning loop	**Interleaved thinking**: reason between tool calls

Mixture of Experts: doing more with less

The core idea

In a standard (dense) transformer, every parameter activates for every token. In a **Mixture of Experts** (MoE) model (Shazeer et al., 2017), each layer contains *N* **expert** sub-networks (FFNs), but only *k* are activated per token:

1. A small **router** network scores each expert for the current token
2. The **top-*k*** experts are selected (e.g., 2 of 8, or 8 of 256)
3. Only selected experts compute; the rest are skipped entirely
4. Outputs are combined as a **weighted sum** based on router scores



Total: 46.7B params (8 experts) — Active per token: 12.9B (2 of 8 selected)

Why this matters

Model	Total params	Active per token	Efficiency
Mixtral 8x7B	46.7B	12.9B (2/8 experts)	3.6×
DeepSeek-V3	671B	37B (8/256 experts)	18×
Llama 4 Maverick	400B	17B (1/128 experts)	23×

Efficiency = total / active — how much more knowledge the model stores vs. what it computes per token. MoE was first proposed by Shazeer et al. (2017) and refined by Fedus et al. (2022).

The frontier landscape: early 2026

Where we stand

Model	Developer	Key capability
Claude Opus 4.6	Anthropic	1M context, adaptive thinking, visible reasoning
GPT-5 / 5.2	OpenAI	Native multimodal, 100% AIME 2025 (GPT-5.2)
Gemini 3.1 Pro	Google	1M context, built-in thinking, 48.4% HLE
DeepSeek-R1	DeepSeek	Open-weight reasoning, 671B MoE, $5.5M training
Llama 4 Maverick	Meta	Open-weight, 400B MoE, 1M context
Qwen 3	Alibaba	81.5% AIME 2025 (235B base), open-weight

The MoE + reasoning convergence

Nearly every frontier model now uses both **reasoning capabilities** and **Mixture of Experts**. DeepSeek-V3 trained for ~$5.6M — roughly 1/20th of GPT-4's estimated cost — proving that reasoning (RL) + efficient architecture (MoE) is a winning combination.

Benchmark saturation and the reasoning gap

Where thinking helps — and where it doesn't

Benchmark	What it measures	Best	Human	Status
MATH-500	500 competition-level math problems	99.4%	—	Saturated
AIME 2025	15-question high school math competition	100%	—	**Saturated**
GPQA Diamond	PhD-level science questions	94.3%	~65%	Near-saturated
SWE-bench Verified	Real GitHub issue resolution	80.9%	—	Active
ARC-AGI-2	Novel visual reasoning patterns	**~54%**	**60%**	**Closing**
Humanity's Last Exam	Expert cross-domain questions	**48.4%**	**~90%**	**Not saturated**

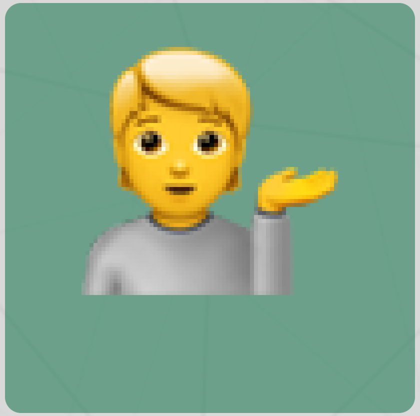
The key pattern

Thinking dramatically helps on problems that *decompose* into verifiable steps (math, coding). It helps less on problems requiring **novel abstractions** (ARC-AGI-2) or **deep domain expertise** (HLE). More thinking tokens ≠ more understanding — it's more computation *within the same learned representations*. ARC-AGI-2 has seen rapid recent progress (~54%, up from 4.4% in late 2024), though a gap to human performance remains.

Questions?

[Email]

[Discord]



[Office hours]

Up next...

Agents, tools, and the agentic era: when LLMs start acting in the world